COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	July 23, 2026	**DATE SUBMITTED:**	July 27, 2026
NAME:	Benj Federic C. De Leon	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 **Linked Lists**

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a doubly linked list in C++

Screenshot

```
#include<iostream>
#include<utility>
class Node{
public:
char data;
Node *next;
};
int main(){
//step 1
Node *head = NULL;
Node *second = NULL;
Node *third = NULL;
Node *fourth = NULL;
Node *fifth = NULL;
Node *last = NULL;
//step 2
head = new Node;
second = new Node;
third = new Node;
fourth = new Node;
fifth = new Node;
last = new Node;
//step 3
head->data = 'C';
head->next = second;
```

	``
	second->data = 'P';
	second->next = third;
	third->data = 'E';
	third->next = fourth;
	fourth->data = '0';
	fourth->next = fifth;
	fifth->data = '1';
	fifth->next = last;
	//step 4
	last->data = '0';
	last->next = nullptr;
	}
	``

Discussion	This first version builds the linked list by hand: each node (head, second, third, fourth, fifth, last) is created separately with new, then its data and next pointer are set one line at a time. It shows the basic idea of a linked list, where each node points to the next, but it is not reusable since there is no function yet for creating nodes or adding/removing them automatically.

Table 3-1. Output of Initial/Simple Implementation

Operation	```
	// 1ST STEP (FUNCTION): Create a new node
	template <typename T>
	doubleList<T>* new_node(T newData) {
	    // dynamically allocate memory for a new node
	    doubleList<T>* newNode = new doubleList<T>;
	
	    // Store data in the node:
	    newNode->data = newData;
	
	    // since its a new node, it will point to null
	    newNode->next = nullptr;
	    newNode->prev = nullptr;
	
	    return newNode;
	}
	``

Traversal	```
	// TRAVERSAL (forward and reverse)
	template <typename T>
	``

```
void dllTraverse(doubleList<T>* currentNode) {
    doubleList<T>* tail = nullptr;

    // traversal to the end
    while (currentNode != nullptr) {
        // print the data
        std::cout << currentNode->data << " ";
        // save the current node as the last visited node
        tail = currentNode;
        // move to the next node
        currentNode = currentNode->next;
    }
    std::cout << std::endl;

    // reverse traversal
    std::cout << "Reverse traversal \n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
```

Insertion at head

```
// INSERTION AT THE HEAD
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead)
{
    // create a new node
    doubleList<T>* newNode = new_node(newData);

    // SET the new node to point to the current head
    newNode->next = *currentHead;

    // current head will be pointed to the new node
    if (*currentHead != nullptr) {
        (*currentHead)->prev = newNode;
    }
```

	``
	    // update the head pointer
	    *currentHead = newNode;
	}
	``

Insertion at any part of the list	```
	// GENERAL INSERTION
	template <typename T>
	void dllInsertAfter(T targetData, T newData,
	doubleList<T>* currentHead) {
	    while (currentHead != nullptr && currentHead->data != targetData) {
	        currentHead = currentHead->next;
	    }
	
	    if (currentHead == nullptr) {
	        std::cout << "Node not found." << std::endl;
	        return;
	    }
	
	    doubleList<T>* newNode = new_node(newData);
	    newNode->next = currentHead->next;
	    newNode->prev = currentHead;
	
	    if (currentHead->next != nullptr) {
	        currentHead->next->prev = newNode;
	    }
	    currentHead->next = newNode;
	}
	``

Insertion at the end	```
	// INSERTION AT THE END
	template <typename T>
	void dllInsertEnd(T newData, doubleList<T>* currentHead) {
	    // create a new node
	    doubleList<T>* newNode = new_node(newData);
	
	    // traverse until the last node
	    while (currentHead->next != nullptr) {
	        currentHead = currentHead->next;
	``

```
    }

    // connect the last node to the new node
    currentHead->next = newNode;

    // connect the prev node to the new node
    newNode->prev = currentHead;
}
```

Deletion of a node

```
// DELETE
template <typename T>
void dllDelete(T targetData, doubleList<T>** currentHead)
{

    if (*currentHead == nullptr) return;
y
    while (*currentHead != nullptr && (*currentHead)->data
!= targetData) {
        currentHead = &((*currentHead)->next);
    }

    if (*currentHead == nullptr) {
        std::cout << "Node not found." << std::endl;
        return;
    }

    // Track the target node to delete it from memory
safely
    doubleList<T>* toDelete = *currentHead;

    // Link the next node's prev pointer backward
    if (toDelete->next != nullptr) {
        toDelete->next->prev = toDelete->prev;
    }

    // Bypass the deleted node forward
    *currentHead = toDelete->next;
```

```
    delete toDelete;
}
```

Table 3-2. Code for the List Operations

a.	**Source Code**	`// Step a: Traverse initial list`
		`std::cout << "a. Initial List traversal: ";`
		`sllTraverse(head); // CPE010`

	Console	`a. Initial List traversal: CPE010` Analysis: This shows the starting list before any changes. Traversing from head prints each node's data in order, so we can see the original sequence.

b.	**Source Code**	`// Step b: Insert 'G' at the head`
		`sllInsertHead('G', &head);`
		`std::cout << "b. Insert 'G' at start: ";`
		`sllTraverse(head); // GCPE101`

	Console	`b. Insert 'G' at start: GCPE101` Analysis: Inserting 'G' at the head creates a new node and makes it the new head, with its next pointer pointing to the old head. The new item now appears first in the traversal.

c.	**Source Code**	`// Step c: Insert 'E' after node 'P'`
		`sllInsertAfter('P', 'E', head);`
		`std::cout << "c. Insert 'E' after 'P': ";`

		`sllTraverse(head); // GCPEE101`
	Console	`c. Insert 'E' after 'P': GCPEE101` Analysis: Inserting 'E' after node 'P' adds a new node in the middle of the list by relinking the next pointers, so the list grows without moving any existing nodes.

d.	**Source Code**	`// Step d: Delete node 'C'`
		`sllDelete('C', &head);`
		`std::cout << "d. After deleting 'C': ";`
		`sllTraverse(head); // GPEE101`

	Console	`d. After deleting 'C': GPEE101` Analysis: Deleting node 'C' removes it from the list by connecting the node before it directly to the node after it, then freeing its memory. The traversal now skips the removed data.

e.	**Source Code**	`sllDelete('P', &head);`
		`std::cout << "e. After deleting 'P': ";`
		`sllTraverse(head); // GEE101`

	Console	`e. After deleting 'P': GEE101` Analysis: Deleting node 'P' works the same way, removing it and reconnecting its neighbors. This shows the delete function works no matter which node is targeted.
f.	**Source Code**	`// Step f: Final output display` `std::cout << "f. Final List output: ";`

		`sllTraverse(head); // GEE101`
	Console	`f. Final List output: GEE101` Analysis: This is the final traversal after all insertions and deletions. It confirms the list correctly reflects every change made in the earlier steps.

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot/s	Analysis

`// a. TRAVERSAL`
`template <typename T>`
`void sllTraverse(singleList<T>* currentNode) {`
`  while (currentNode != nullptr) {`
`  std::cout << currentNode->data;`
`  currentNode = currentNode->next;`
`  }`
`  std::cout << std::endl;`
`}`
	This function visits every node in a singly linked list one by one, printing its data until it reaches the end.

`// b. INSERTION AT THE HEAD`
`template <typename T>`
`void sllInsertHead(T newData, singleList<T>** currentHead) {`
`  singleList<T>* newNode = new_snode(newData);`
`  newNode->next = *currentHead;`
`  *currentHead = newNode;`
	This code creates a new node, links its next pointer to the existing start of the list, and then updates the head pointer so the new node becomes the list's new first element .

``
// c. GENERAL INSERTION (AFTER A TARGET NODE)
template <typename T>
void sllInsertAfter(T targetData, T newData, singleList<T>* currentHead) {
    while (currentHead != nullptr && currentHead->data != targetData) {
        currentHead = currentHead->next;
    }
 	
    singleList<T>* newNode = new_snode(newData);
    newNode->next = currentHead->next;
    currentHead->next = newNode;
``
	This code searches through the linked list to find a specific target node, attaches a new node right after it, and updates the references so the rest of the list stays connected.

``
// INSERTION AT THE END
template <typename T>
void sllInsertEnd(T newData, singleList<T>** currentHead) {
    singleList<T>* newNode = new_snode(newData);
    if (*currentHead == nullptr) {
        *currentHead = newNode;
        return;
    }
 	
    singleList<T>* current = *currentHead;
    while (current->next != nullptr) {
        current = current->next;
    }
``
	This code attaches a new node to the very tail of the linked list by traversing to the last node and linking it, or simply sets the new node as the head if the list is empty.

`current->next = newNode;`	

`/ d. DELETE A NODE`
`template <typename T>`
`void sllDelete(T targetData, singleList<T>** currentHead) {`
`if (*currentHead == nullptr) return;`
 	
`singleList<T>* current = *currentHead;`
`singleList<T>* prev = nullptr;`
 	
`// Traverse to search for target`
`while (current != nullptr && current->data != targetData) {`
`prev = current;`
`current = current->next;`
`}`
 	
`prev->next = current->next;`
`delete current;`
`}`
	This code searches the list for a node with the target data, bypasses it by connecting the previous node directly to the next one, and frees the memory of the removed node.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple

players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

main_playlist.cpp	

```cpp
#include <iostream>
#include <limits>
#include "playlist_CLL.h"

int main() {
    // HEAD of the circular doubly linked list, and a separate pointer
    // that tracks whichever song is "currently playing"
    PlaylistNode<std::string>* head = nullptr;
    PlaylistNode<std::string>* currentTrack = nullptr;

    int choice;
    do {
        printMenu();
        std::cin >> choice;

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        switch (choice) {
```

```
            case 1: handleInsert(&head, &currentTrack);
break;
            case 2: handleRemove(&head, &currentTrack);
break;
            case 3: handlePlayAll(head); break;
            case 4: handleNext(&currentTrack); break;
            case 5: handlePrev(&currentTrack); break;
            case 6: handleShowCurrent(currentTrack);
break;
            case 0: std::cout << "Exiting
playlist...\n"; break;
            default: std::cout << "Invalid choice, try
again.\n";
        }
    } while (choice != 0);

    cleanupPlaylist(&head);
    return 0;
}
```

playlist_CLL.h

```
#ifndef PLAYLIST_CLL_H
#define PLAYLIST_CLL_H

#include <iostream>
#include <string>

template <typename T>
class PlaylistNode {
public:
    T data;
    PlaylistNode<T>* next = nullptr;
    PlaylistNode<T>* prev = nullptr;
};

// 1ST STEP (FUNCTION): Create a new node
template <typename T>
```

```
PlaylistNode<T>* new_node(T newData) {
    PlaylistNode<T>* newNode = new PlaylistNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}

// TRAVERSAL (Plays/Prints all songs in a circular loop
once)
template <typename T>
void playlistTraverse(PlaylistNode<T>* currentHead) {

    PlaylistNode<T>* temp = currentHead;
    std::cout << "--- Playing All Songs ---\n";
    do {
        std::cout << "Playing: " << temp->data <<
std::endl;
        temp = temp->next;
    } while (temp != currentHead);
    std::cout << "-------------------------\n";
}

// INSERTION AT THE END / ADD SONG
template <typename T>
void playlistInsert(T newData, PlaylistNode<T>**
currentHead) {
    PlaylistNode<T>* newNode = new_node(newData);

    if (*currentHead == nullptr) {
        *currentHead = newNode;
        newNode->next = newNode;
        newNode->prev = newNode;
        return;
    }

    PlaylistNode<T>* tail = (*currentHead)->prev;
```

```
    tail->next = newNode;
    newNode->prev = tail;
    newNode->next = *currentHead;
    (*currentHead)->prev = newNode;
}

// DELETE A SONG
template <typename T>
void playlistDelete(T targetData, PlaylistNode<T>**
currentHead) {

    PlaylistNode<T>* current = *currentHead;
    bool found = false;

    // Search for target node circularly
    do {
        if (current->data == targetData) {
            found = true;
            break;
        }
        current = current->next;
    } while (current != *currentHead);

    // Single node case
    if (current->next == current && current->prev ==
current) {
        delete current;
        *currentHead = nullptr;
        return;
    }

    // Update predecessor and successor
    current->prev->next = current->next;
    current->next->prev = current->prev;

    // Update head if head is being deleted
```

```
    if (current == *currentHead) {
        *currentHead = current->next;
    }

    delete current;
}

// NAVIGATION: NEXT SONG
template <typename T>
void playlistNext(PlaylistNode<T>** currentTrack) {
    if (*currentTrack != nullptr) {
        *currentTrack = (*currentTrack)->next;
        std::cout << "Now Playing (Next): " <<
(*currentTrack)->data << std::endl;
    }
}

// NAVIGATION: PREVIOUS SONG
template <typename T>
void playlistPrev(PlaylistNode<T>** currentTrack) {
    if (*currentTrack != nullptr) {
        *currentTrack = (*currentTrack)->prev;
        std::cout << "Now Playing (Prev): " <<
(*currentTrack)->data << std::endl;
    }
}

//
============================================================
====
// DRIVER / MENU HELPERS (used by main_playlist.cpp)
// "inline" is needed here because these are plain
(non-template)
// functions defined in a header - without it, including
this
// header in more than one .cpp file would cause a
linker error.
```

```
//
============================================================
====

inline void printMenu() {
    std::cout << "\n===== PLAYLIST MENU =====\n";
    std::cout << "1. Insert Song\n";
    std::cout << "2. Remove Song\n";
    std::cout << "3. Play All Songs\n";
    std::cout << "4. Next Song\n";
    std::cout << "5. Previous Song\n";
    std::cout << "6. Show Current Song\n";
    std::cout << "0. Exit\n";
    std::cout << "========================\n";
    std::cout << "Choice: ";
}

inline void handleInsert(PlaylistNode<std::string>**
head, PlaylistNode<std::string>** currentTrack) {
    std::cout << "Enter song title: ";
    std::string title;
    std::getline(std::cin, title);

    playlistInsert(title, head);

    // "first song ever added" check, done via switch
instead of if/else
    switch (*currentTrack == nullptr) {
        case true:
            *currentTrack = *head;
            break;
        case false:
            break;
    }
    std::cout << "Added: " << title << std::endl;
}
```

```
inline void handleRemove(PlaylistNode<std::string>**
head, PlaylistNode<std::string>** currentTrack) {
    switch (*head == nullptr) {
        case true:
            std::cout << "Playlist is empty.\n";
            break;

        case false: {
            std::cout << "Enter song title to remove: ";
            std::string title;
            std::getline(std::cin, title);

            // If the song being removed is the one
currently playing,
            // move currentTrack off it BEFORE deleting,
so it never
            // points to freed memory.
            switch ((*currentTrack != nullptr) &&
((*currentTrack)->data == title)) {
                case true:
                    switch ((*currentTrack)->next !=
*currentTrack) {
                        case true:
                            *currentTrack =
(*currentTrack)->next;
                            break;
                        case false:
                            *currentTrack = nullptr;
                            break;
                    }
                    break;
                case false:
                    break;
            }

            playlistDelete(title, head);
```

```
            switch (*head == nullptr) {
                case true:
                    *currentTrack = nullptr;
                    break;
                case false:
                    break;
            }
            std::cout << "Removed (if it existed): " <<
title << std::endl;
            break;
        }
    }
}

inline void handlePlayAll(PlaylistNode<std::string>*
head) {
    switch (head == nullptr) {
        case true:
            std::cout << "Playlist is empty.\n";
            break;
        case false:
            playlistTraverse(head);
            break;
    }
}

inline void handleNext(PlaylistNode<std::string>**
currentTrack) {
    switch (*currentTrack == nullptr) {
        case true:
            std::cout << "Playlist is empty.\n";
            break;
        case false:
            playlistNext(currentTrack);
            break;
    }
}
```

```
inline void handlePrev(PlaylistNode<std::string>**
currentTrack) {
    switch (*currentTrack == nullptr) {
        case true:
            std::cout << "Playlist is empty.\n";
            break;
        case false:
            playlistPrev(currentTrack);
            break;
    }
}

inline void handleShowCurrent(PlaylistNode<std::string>*
currentTrack) {
    switch (currentTrack == nullptr) {
        case true:
            std::cout << "Nothing is currently
playing.\n";
            break;
        case false:
            std::cout << "Current Song: " <<
currentTrack->data << std::endl;
            break;
    }
}

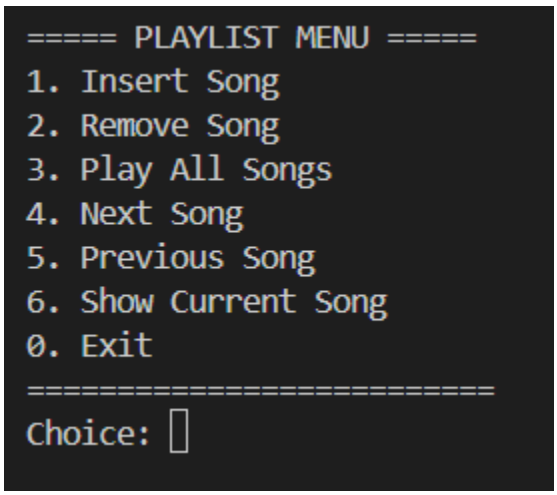
inline void cleanupPlaylist(PlaylistNode<std::string>**
head) {
    switch (*head == nullptr) {
        case true:
            break;

        case false: {
            PlaylistNode<std::string>* current = *head;
            current->prev->next = nullptr; // break the
circular connection to stop the loop
```

```
            while (current != nullptr) {
                PlaylistNode<std::string>* temp =
current;
                current = current->next;
                delete temp;
            }
            *head = nullptr;
            break;
        }
    }
}

#endif
```

OUTPUT:



Code analysis:

The playlist program uses a circular doubly linked list so that every song points to a next and a previous song, and the last song connects back to the first one. This makes it easy to move forward or backward through the songs without ever reaching a dead end, which fits a music player where the playlist loops. Inserting a song attaches a new node between the current tail and head, while deleting a song relinks its neighboring nodes so

the loop stays unbroken.

Conclusion:

This activity helped us understand how linked lists work differently from arrays, since nodes can be added or removed without shifting other elements. We learned the difference between singly linked lists (one direction), doubly linked lists (two directions), and circular linked lists (no end), and saw how a circular doubly linked list is a natural fit for a real-world application like a playlist that loops through songs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, formatting, summarization, explanation of the concepts, idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I still manage to analyze the code first and re-checking it to the concept I believe is right. And applying the lessons and ideas our prof discussed.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.